

LangChain-Core

Architecture & Source Code Analysis

Version 0.1.16 | Deep Dive into Core Abstractions

1. Project Overview & Module Structure
2. Runnable System & LCEL (Core Flow Diagrams)
3. Message System Architecture
4. Language Model Abstraction
5. Callback & Tracing System (Sequence Diagrams)
6. Prompt / OutputParser / Tool / Retriever Design
7. Key Class Reference Table

1. Project Overview & Module Structure

langchain-core is the foundational package of the LangChain ecosystem. It defines base abstractions, interfaces, and protocols that all other packages depend on. No third-party integrations are included — dependencies are kept purposefully lightweight.

1.1 Module Overview

Module	Description	Key Classes
runnables/	LCEL protocol & composition	Runnable, RunnableSequence, RunnableParallel, RunnableLambda
messages/	Message types & streaming	BaseMessage, AIMessage, HumanMessage, ToolMessage
language_models /	LLM/ChatModel abstractions	BaseLanguageModel, BaseChatModel, BaseLLM
callbacks/	Event-driven callback system	CallbackManager, RunManager, BaseCallbackHandler
tracers/	Execution tracing & observability	BaseTracer, LangChainTracer, LogStreamHandler
prompts/	Prompt template system	ChatPromptTemplate, PromptTemplate, MessagesPlaceholder
output_parsers/	Output parsing & validation	BaseOutputParser, JsonOutputParser, PydanticOutputParser
tools/	Tool abstraction	BaseTool, StructuredTool, tool decorator
documents/	Document types	Document, BaseDocumentTransformer, BaseDocumentCompressor
vectorstores/	Vector store interface	VectorStore, VectorStoreRetriever
embeddings/	Embedding interface	Embeddings (ABC)
load/	Serialization/deserialization	Serializable, dumpd(), load()

1.2 Overall Architecture Diagram

LangChain-Core Overall Architecture

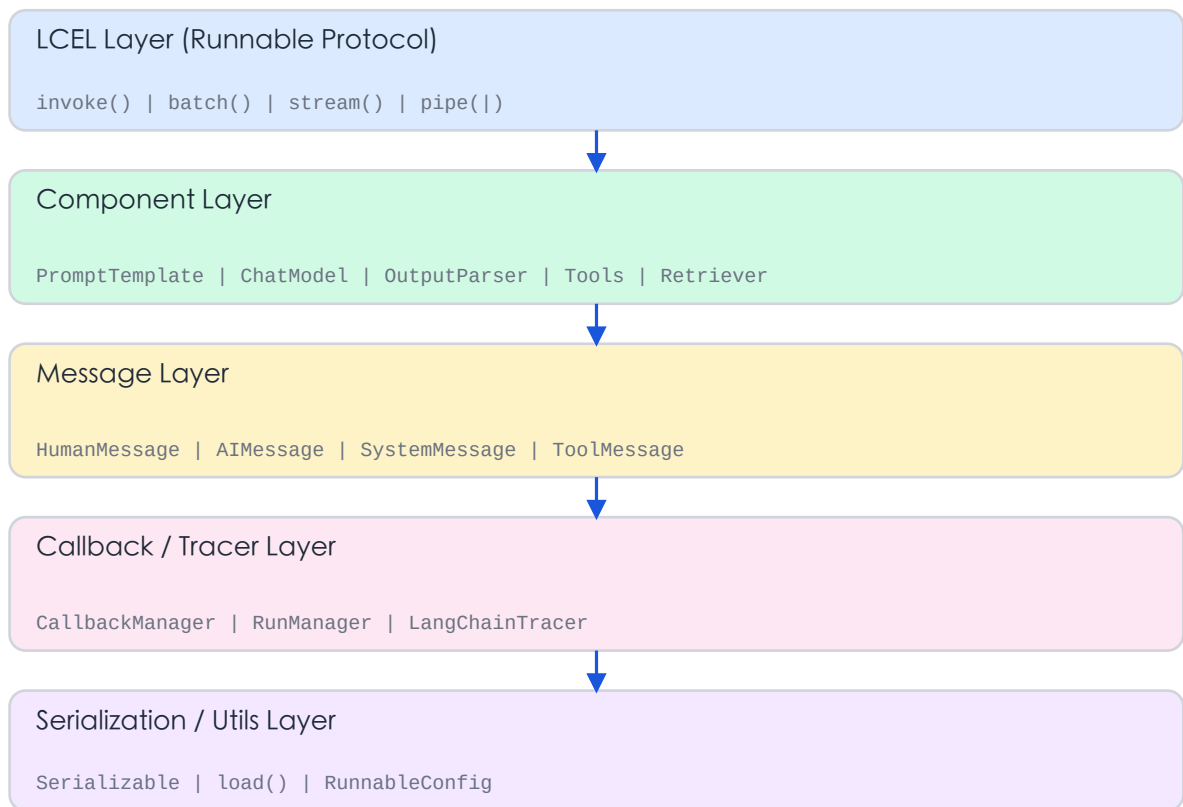
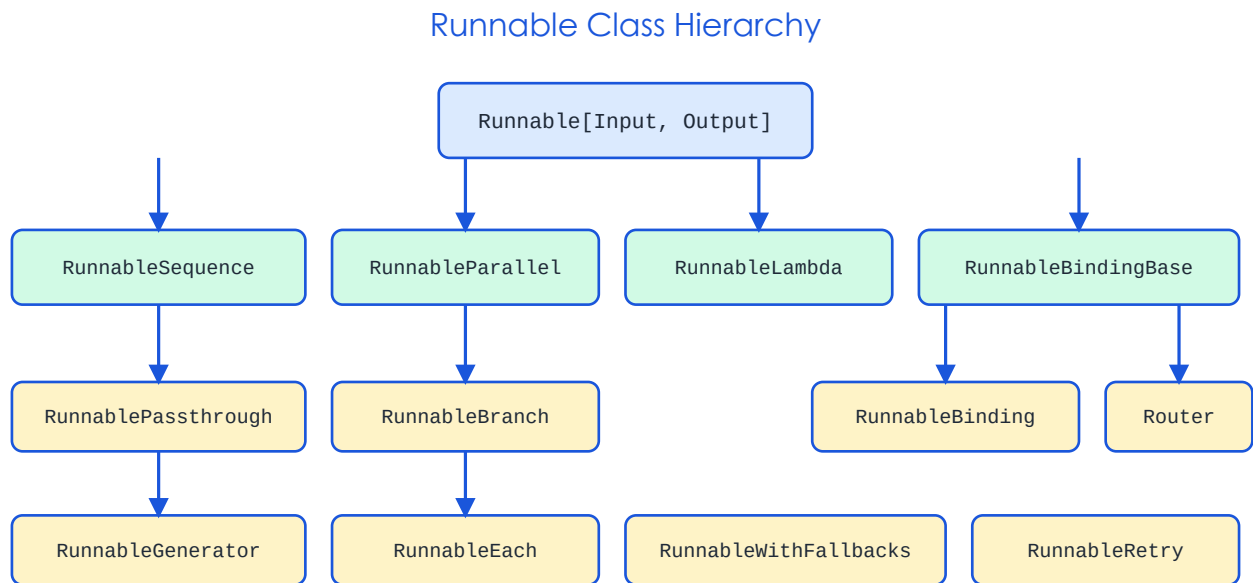


Fig 1.1 — LangChain-Core Layered Architecture

2. Runnable System & LCEL

The Runnable protocol is the heart of LangChain-Core. Every component — prompts, models, parsers, tools, retrievers — implements this protocol, making them composable via the LangChain Expression Language (LCEL).

2.1 Runnable Class Hierarchy



Runnable Protocol (Key Methods):

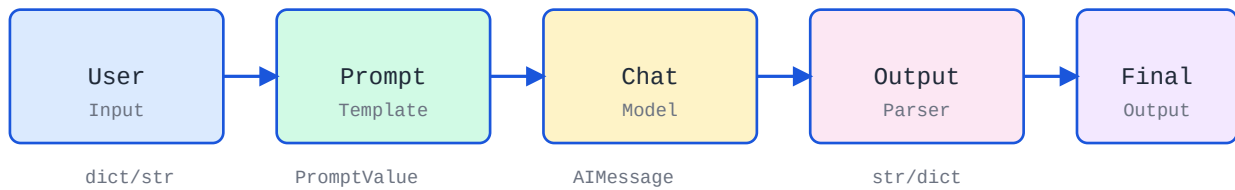
```
invoke(input, config) -> Output          # Single invocation
batch(inputs, config) -> list[Output]      # Batch processing
stream(input, config) -> Iterator[Output] # Streaming output
ainvoke / abatch / astream                 # Async variants
pipe(other) / __or__(other)               # LCEL chain composition
bind(**kwargs) / with_config(config)      # Configuration binding
```

Fig 2.1 — Runnable Class Hierarchy and Protocol Methods

2.2 LCEL Pipeline Data Flow

LCEL allows composing Runnables using the pipe operator (`|`). Under the hood, `a | b` creates a `RunnableSequence([a, b])`. Data flows through each step, with automatic type coercion at each boundary.

LCEL Pipeline Data Flow



LCEL Expression:

```
chain = prompt_template | chat_model | output_parser
```

Fig 2.2 — Data flow through an LCEL pipeline

2.3 Key Runnable Types

Runnable Type	Purpose	Creation
RunnableSequence	Sequential chain: A -> B -> C	a b c
RunnableParallel	Parallel execution with dict output	RunnableParallel(key1=r1, key2=r2)
RunnableLambda	Wrap any function as Runnable	RunnableLambda(fn) or @chain
RunnablePassthrough	Pass input through, optionally assign	RunnablePassthrough.assign(key=fn)
RunnableBranch	Conditional routing	RunnableBranch((cond1, r1), ..., default)
RunnableBinding	Bind config/kwargs to a Runnable	runnable.bind(stop=['\n'])
RunnableWithFallbacks	Fallback on errors	runnable.with_fallbacks([fallback1])
RunnableRetry	Retry with backoff	runnable.with_retry(stop_after=3)
RunnableGenerator	Wrap a generator function	For custom streaming transforms

2.4 RunnableSequence.invoke() Internal Flow

```
def invoke(self, input, config=None, **kwargs):
    # Patch config with callbacks for this chain
    config = ensure_config(config)
    callback_manager = get_callback_manager_for_config(config)
    run_manager = callback_manager.on_chain_start(
        dumpd(self), input, name=config.get('run_name') or self.get_name()
    )
    try:
        # Execute each step sequentially
        for i, step in enumerate(self.steps):
            # Each step gets a child callback manager
```

```
input = step.invoke(
input,
patch_config(config, callbacks=run_manager.get_child(f'seq:step:{i+1}'))
)
except Exception as e:
run_manager.on_chain_error(e)
raise
run_manager.on_chain_end(input)
return input
```

3. Message System Architecture

The Message system is the standardized communication format between users, AI models, and tools. It provides a provider-agnostic abstraction layer over different LLM APIs, with full support for multimodal content, streaming, and serialization.

3.1 Message Class Hierarchy

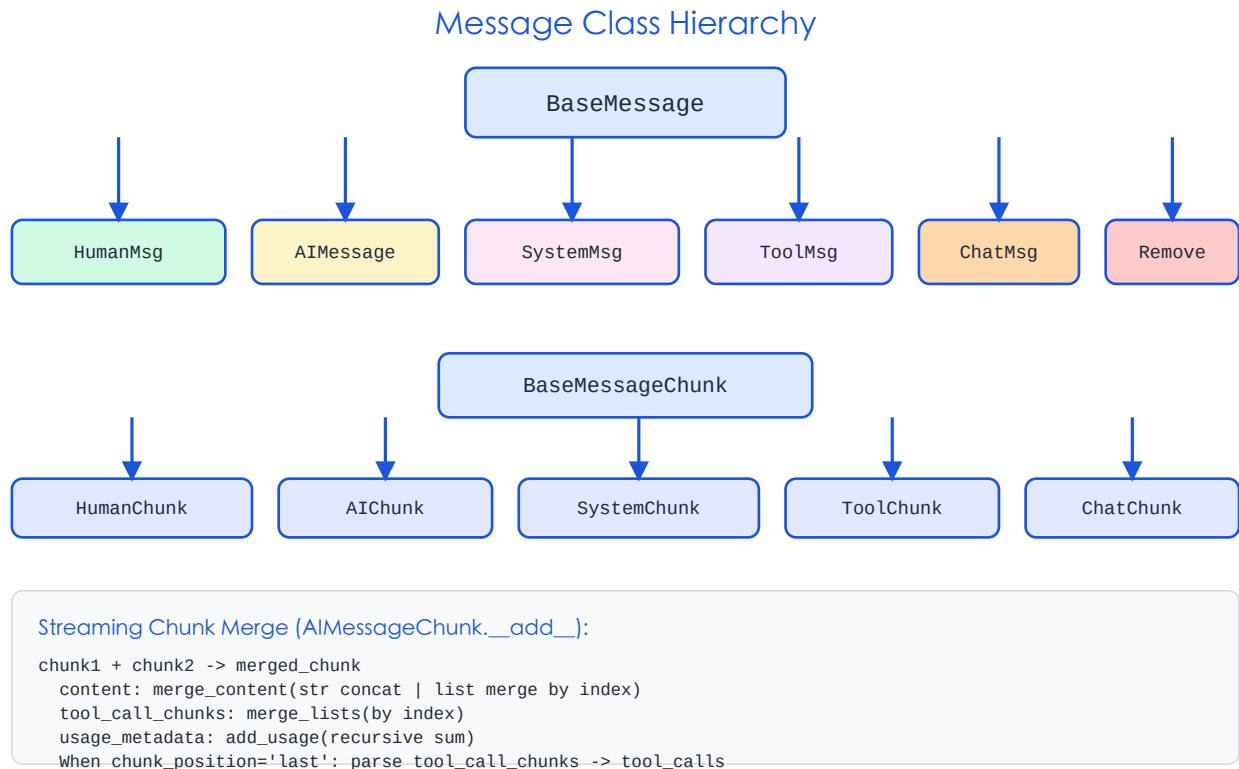


Fig 3.1 — Message and Chunk class hierarchy with streaming merge

3.2 Message Types

Message Type	type Field	Key Extra Fields	Use Case
HumanMessage	"human"	—	User input to the model
AIMessage	"ai"	tool_calls, usage_metadata, invalid_tool_calls	Model response
SystemMessage	"system"	—	System instructions / behavior prompt
ToolMessage	"tool"	tool_call_id, artifact, status	Tool execution result
FunctionMessage	"function"	name (deprecated)	Legacy function call result
ChatMessage	"chat"	role: str	Custom role messages

Message Type	type Field	Key Extra Fields	Use Case
RemoveMessage	"remove"	id (required)	Delete message by ID (LangGraph)

3.3 BaseMessage Core Fields

Field	Type	Description
content	str list[str dict]	Message content — string or multimodal content blocks
additional_kwargs	dict	Extra payload (e.g., raw tool_calls from provider)
response_metadata	dict	Response metadata (headers, logprobs, model_name, provider)
type	str	Message type discriminator for deserialization
name	str None	Optional human-readable name
id	str None	Optional unique identifier

3.4 Content Block Type System

LangChain v1 introduces a provider-agnostic ContentBlock type system for standardizing multimodal content. Each content block is a TypedDict with a `type` discriminator.

Block Type	type Value	Key Fields
TextContentBlock	"text"	text, annotations
ReasoningContentBlock	"reasoning"	text (model thinking process)
ImageContentBlock	"image"	url / base64 / file_id, mime_type
AudioContentBlock	"audio"	url / base64, mime_type
VideoContentBlock	"video"	url / base64, mime_type
FileContentBlock	"file"	url / base64, mime_type
ToolCall	"tool_call"	name, args, id
ToolCallChunk	"tool_call_chunk"	name, args (partial), id, index

4. Language Model Abstraction

Language models are the core AI components. LangChain provides two model hierarchies: BaseLLM for traditional text completion (legacy) and BaseChatModel for modern chat-based models (recommended). Both inherit from BaseLanguageModel which is itself a Runnable.

4.1 Language Model Class Hierarchy

Class	Input	Output	Status
BaseLanguageModel	LanguageModelInput	str AIMessage	Abstract base
BaseLLM	str / PromptValue	str	Legacy (text completion)
LLM	str	str (via _call)	Simplified LLM
BaseChatModel	list[BaseMessage]	AIMessage	Recommended (modern)
SimpleChatModel	list[BaseMessage]	str (via _call)	Simplified chat model

4.2 BaseChatModel Key Methods

Method	Must Implement?	Description
<code>_generate(messages, stop, run_manager, **kw)</code>	Required	Core generation logic, returns ChatResult
<code>_llm_type</code> (property)	Required	Model type identifier string
<code>_stream(messages, stop, run_manager, **kw)</code>	Optional	Streaming output, yields ChatGenerationChunk
<code>_agenerate(...)</code>	Optional	Async generation (defaults to <code>run_in_executor</code>)
<code>_astream(...)</code>	Optional	Async streaming
<code>bind_tools(tools, tool_choice, **kw)</code>	Optional*	Bind tools for function calling
<code>with_structured_output(schema, **kw)</code>	Optional*	Returns Runnable that outputs structured data

* Required by partner packages (e.g., langchain-openai, langchain-anthropic)

4.3 ChatModel.invoke() Sequence Diagram

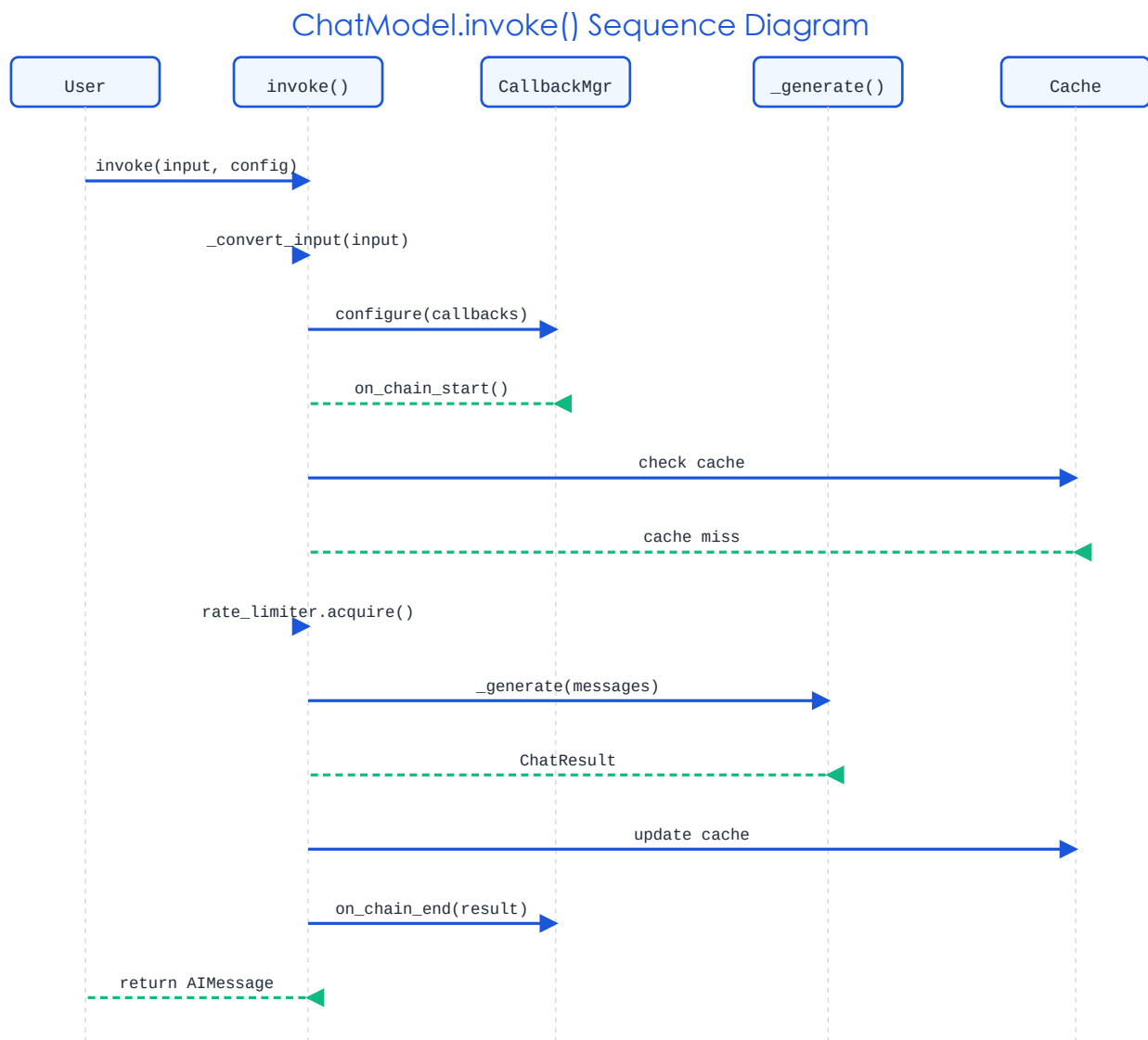
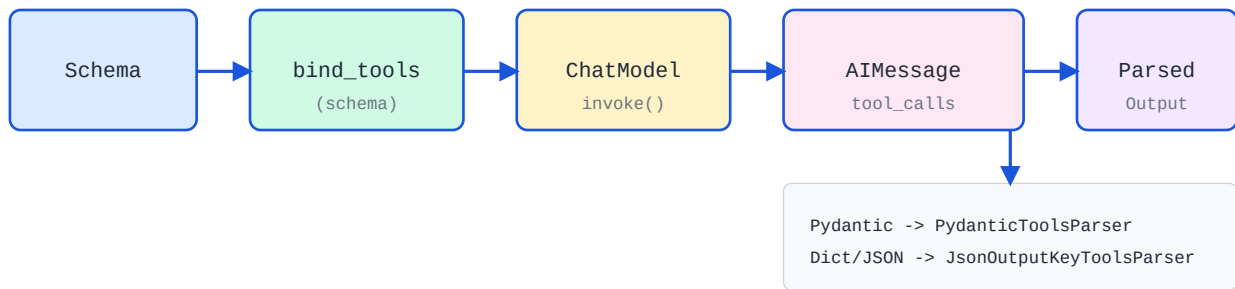


Fig 4.1 — Complete sequence of ChatModel.invoke() execution

4.4 Tool Calling & Structured Output

with_structured_output() Pipeline



Internal Pipeline:

```
llm = self.bind_tools([schema], tool_choice='any')
return llm | output_parser # RunnableSequence
```

Fig 4.2 — `with_structured_output()` internal pipeline

5. Callback & Tracing System

The callback system is a multi-layer Observer pattern that provides observability, logging, and tracing throughout the execution chain. Tracers are specialized callback handlers that maintain a tree of Run objects for full execution tracing.

5.1 Callback Architecture Layers

Layer	Key Classes	Responsibility
Config Layer	RunnableConfig	Carries callbacks, tags, metadata through the chain
Manager Layer	CallbackManager, AsyncCallbackManager	Event dispatching & handler lifecycle
Run Manager Layer	RunManager, CallbackManagerForChainRun, etc.	Bound to specific run, parent-child propagation
Handler Layer	BaseCallbackHandler, AsyncCallbackHandler	Event consumers (Observer interface)
Tracer Layer	BaseTracer, LangChainTracer, LogStreamHandler	Specialized handlers with Run tree management

5.2 Callback Handler Interface

Mixin	Events Provided
CallbackManagerMixin	on_llm_start, on_chat_model_start, on_chain_start, on_tool_start, on_retriever_start
LLMManagerMixin	on_llm_new_token, on_llm_end, on_llm_error
ChainManagerMixin	on_chain_end, on_chain_error, on_agent_action, on_agent_finish
ToolManagerMixin	on_tool_end, on_tool_error
RetrieverManagerMixin	on_retriever_end, on_retriever_error
RunManagerMixin	on_text, on_retry, on_custom_event

5.3 Callback Propagation in Runnable Chain

Callback Propagation in Runnable Chain

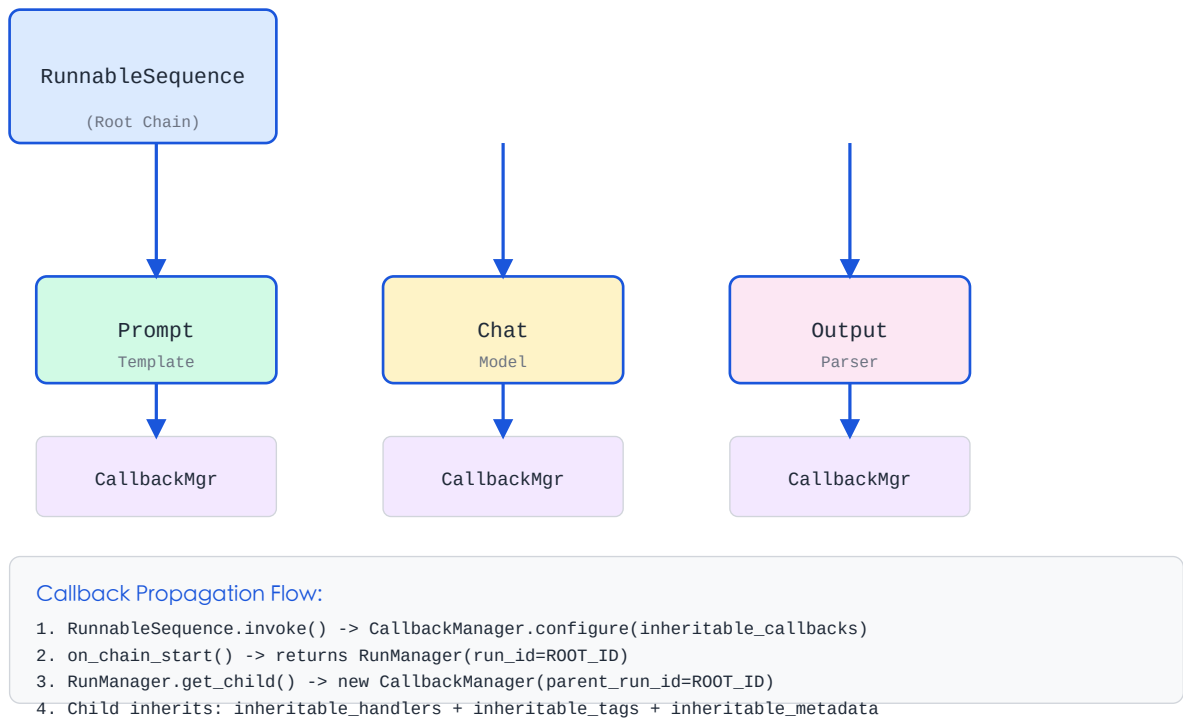


Fig 5.1 — How callbacks propagate through the Runnable execution tree

5.4 Streaming Flow Sequence

Streaming Flow: ChatModel.stream()

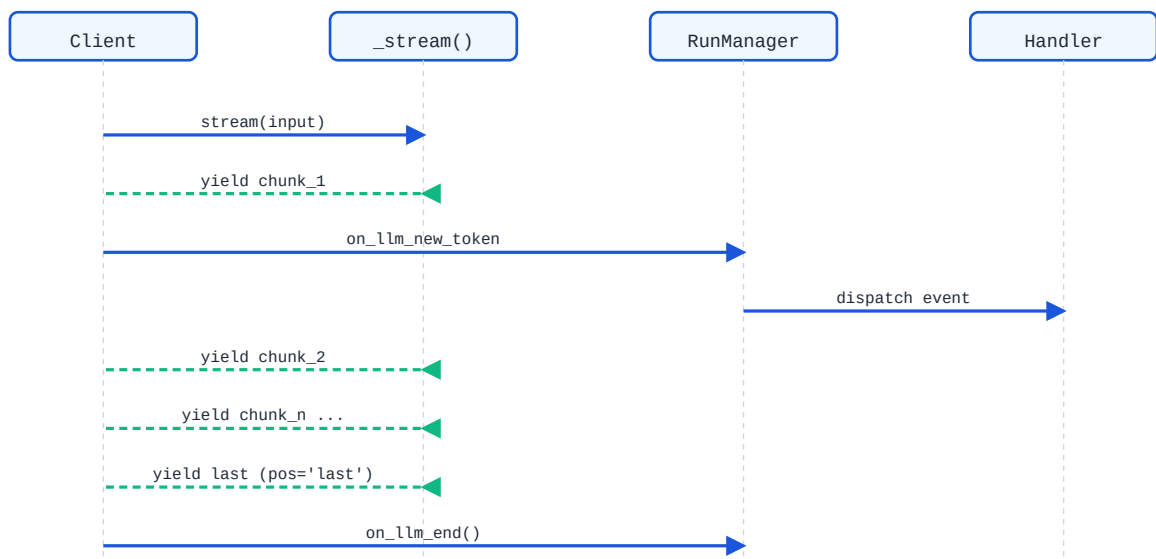


Fig 5.2 — Streaming token-by-token sequence diagram

5.5 Key Tracer Implementations

Tracer	Description	Activation
LangChainTracer	Sends run data to LangSmith via POST/PATCH	LANGCHAIN_TRACING_V2=true
ConsoleCallbackHandler	Prints colored breadcrumb traces to stdout	debug=True
LogStreamCallbackHandler	Drives astream_log() API via async Queue	runnable.astream_log()
_AstreamEventsCallbackHandler	Drives astream_events() API	runnable.astream_events()
RunCollectorCallbackHandler	Collects root Run objects to list	with collect_runs() as cb:
UsageMetadataCallbackHandler	Tracks token usage across calls	with get_usage_metadata_callback():
RootListenersTracer	User-defined callbacks on root run	runnable.with_listeners(on_end=fn)

6. Prompt / OutputParser / Tool / Retriever

6.1 Prompt Template System

Prompt templates convert user inputs into model-ready prompts. All templates are `Runnable`s that accept a dict input and produce a `PromptValue`.

Class	Description	Example
<code>BasePromptTemplate</code>	Abstract base (<code>Runnable[dict, PromptValue]</code>)	—
<code>PromptTemplate</code>	Single string template with f-string variables	<code>PromptTemplate.from_template("Hi {name}")</code>
<code>ChatPromptTemplate</code>	Multi-message template for chat models	<code>ChatPromptTemplate.from_messages([...])</code>
<code>MessagesPlaceholder</code>	Placeholder for message list insertion	<code>MessagesPlaceholder("history")</code>
<code>FewShotPromptTemplate</code>	Template with example selector	Combines examples + template
<code>StructuredPrompt</code>	Generates Pydantic-compatible structured input	<code>StructuredPrompt.from_messages_and_schema(...)</code>
<code>ImagePromptTemplate</code>	Image URL/base64 template	For multimodal prompts

6.2 Output Parser System

Output parsers transform raw model output (string or `AIMessage`) into structured data. They implement the `Runnable` protocol and support streaming via `BaseTransformOutputParser`.

Parser	Output Type	Description
<code>BaseOutputParser[T]</code>	<code>T</code> (generic)	Abstract base — <code>Runnable[str AIMessage, T]</code>
<code>BaseTransformOutputParser[T]</code>	<code>T</code> (generic)	Adds streaming transform support
<code>StrOutputParser</code>	<code>str</code>	Extracts string from message content
<code>JsonOutputParser</code>	<code>dict</code>	Parses JSON from output (supports partial streaming)
<code>PydanticOutputParser</code>	<code>BaseModel</code>	Parses and validates via Pydantic schema
<code>CommaSeparatedListOutputParser</code>	<code>list[str]</code>	Splits comma-separated output
<code>XMLOutputParser</code>	<code>dict</code>	Parses XML output into dict structure

Parser	Output Type	Description
JsonOutputToolsParser	list[dict]	Parses tool_calls from AIMessage
PydanticToolsParser	list[BaseModel]	Parses tool_calls into Pydantic models

6.3 Tool System

Tools represent actions that an AI can invoke. They bridge the gap between model-generated tool calls and actual function execution. All tools are Runnables.

Class / Decorator	Description	Usage
BaseTool	Abstract base — Runnable[str dict, Any]	Subclass for custom tools
StructuredTool	Tool with Pydantic args schema	StructuredTool.from_function(fn, args_schema=...)
Tool (SimpleTool)	Simple single-string-input tool	Tool(name=..., func=..., description=...)
@tool decorator	Convert function to StructuredTool	@tool\ndef search(query: str): ...
create_retriever_tool()	Wrap retriever as a tool	For RAG agent scenarios
InjectedToolArg	Mark args as runtime-injected (hidden from model)	Annotate with InjectedToolArg

6.4 Retriever & VectorStore

Retrievers fetch relevant documents. BaseRetriever is a Runnable[str, list[Document]]. VectorStores provide similarity search with as_retriever() to convert to a retriever.

Class	Type	Key Methods
BaseRetriever	Runnable[str, list[Document]]	invoke(query) -> list[Document], _get_relevant_documents()
VectorStore	ABC	add_documents(), similarity_search(), as_retriever()
VectorStoreRetriever	BaseRetriever	Wraps VectorStore with search_type & search_kwargs
Document	Serializable	page_content: str, metadata: dict, id: str None
Embeddings	ABC	embed_documents(texts), embed_query(text)

7. Key Class Reference

This section provides a comprehensive reference of the most important classes and their relationships in langchain-core.

7.1 Core Design Principles

- **Runnable Protocol as Universal Interface** — Every component implements `invoke/batch/stream`, enabling universal composition via LCEL. The pipe operator (`|`) creates `RunnableSequence` chains.
- **Provider-Agnostic Abstraction** — `BaseMessage` and `ContentBlock` systems abstract away provider-specific formats. Block translators convert between OpenAI, Anthropic, Google, etc.
- **Streaming-First Design** — Every Message has a corresponding Chunk class. The `__add__` protocol enables incremental merging of streamed responses.
- **Observer Pattern for Observability** — The callback system decouples execution from monitoring. Handlers can be injected via config, context variables, or environment variables.
- **Inheritable Configuration** — `RunnableConfig` propagates through the chain. Callbacks, tags, and metadata follow inheritable/non-inheritable semantics.
- **Serializable Everything** — Core classes extend `Serializable` for JSON-compatible serialization. The `type` field enables polymorphic deserialization.

7.2 Complete Class Relationship Map

Base Class	Key Subclasses	Protocol Role
<code>Runnable[I, O]</code>	<code>RunnableSequence</code> , <code>RunnableParallel</code> , <code>RunnableLambda</code> , <code>RunnablePassthrough</code> , <code>RunnableBranch</code>	Universal invocation & composition
<code>RunnableSerializable</code>	<code>BaseLanguageModel</code> , <code>BaseRetriever</code> , <code>BaseTool</code> , <code>BaseOutputParser</code> , <code>BasePromptTemplate</code>	Serializable Runnable
<code>BaseMessage</code>	<code>HumanMessage</code> , <code>AIMessage</code> , <code>SystemMessage</code> , <code>ToolMessage</code> , <code>ChatMessage</code> , <code>RemoveMessage</code>	Chat message abstraction
<code>BaseMessageChunk</code>	<code>AIMessageChunk</code> , <code>HumanMessageChunk</code> , <code>SystemMessageChunk</code> , <code>ToolMessageChunk</code>	Streaming message fragment
<code>BaseLanguageModel</code>	<code>BaseLLM</code> , <code>BaseChatModel</code>	Language model abstraction
<code>BaseCallbackHandler</code>	<code>BaseTracer</code> , <code>StdOutCallbackHandler</code> , <code>StreamingStdOutCallbackHandler</code>	Event observer

Base Class	Key Subclasses	Protocol Role
BaseTracer	LangChainTracer, ConsoleCallbackHandler, LogStreamCallbackHandler	Execution tracing
BasePromptTemplate	PromptTemplate, ChatPromptTemplate, FewShotPromptTemplate	Prompt formatting
BaseOutputParser	StrOutputParser, JsonOutputParser, PydanticOutputParser, XMLOutputParser	Output parsing
BaseTool	StructuredTool, Tool, RetrieverTool	Tool execution

7.3 Key TypedDict Types

TypedDict	Module	Key Fields
RunnableConfig	runnables/config.py	callbacks, tags, metadata, run_name, max_concurrency, recursion_limit
ToolCall	messages/tool.py	name, args (dict), id, type='tool_call'
ToolCallChunk	messages/tool.py	name, args (str partial), id, index, type='tool_call_chunk'
UsageMetadata	messages/ai.py	input_tokens, output_tokens, total_tokens, input_token_details, output_token_details
ContentBlock	messages/content.py	Union of Text/Image/Audio/Video/File/ToolCall blocks
ModelProfile	language_models/model_profile.py	text_inputs, image_inputs, tool_calling, max_input_tokens, etc.
Generation	outputs/generation.py	text, generation_info, type
ChatGeneration	outputs/chat_generation.py	message (BaseMessage), text, generation_info
ChatResult	outputs/chat_result.py	generations: list[ChatGeneration], llm_output
LLMResult	outputs/llm_result.py	generations: list[list[Generation]], llm_output, run

This document was auto-generated by deep analysis of langchain-core v0.1.16 source code. All diagrams and class descriptions are based on actual implementation.